DESIGN DOCUMENT · NOT YET BUILT

# Call management service

How our customer phone system will work, from first principles to every feature: VoIP, inbound and outbound calls, caller identification from the loan system, the Yeastar PBX, security, compliance and the rollout.

Companion to the detailed Markdown documents in `backend/call-management/docs/`. Where a Yeastar API detail is marked `verify`, it must be checked against Yeastar's developer guide for our PBX edition before coding.

### Contents


1. The goal and the big picture
2. VoIP from scratch: signalling, media, codecs, WebRTC, trunks, NAT, quality
3. Architecture: the pieces and how they trust each other
4. Yeastar: what we use and what is open source
5. Inbound calls, step by step
6. Caller identification with the LMS, and unknown callers
7. Outbound calls and campaigns
8. Every call feature in detail
9. The call lifecycle (states)
10. Data, API and live events
11. Security and compliance (Kenya)
12. Operations: sizing, monitoring, failures
13. Rollout, decisions needed, and what was forgotten

# 1. The goal and the big picture

Agents call customers on their personal phones and take customer calls **from inside Hub**, with no desk phone. Every call is matched to the right customer from the loan system the moment it rings, is recorded for audit, routed to the right team, measured, and visible to supervisors. Only the people the organization allows can use any of it, and every sensitive action is logged.

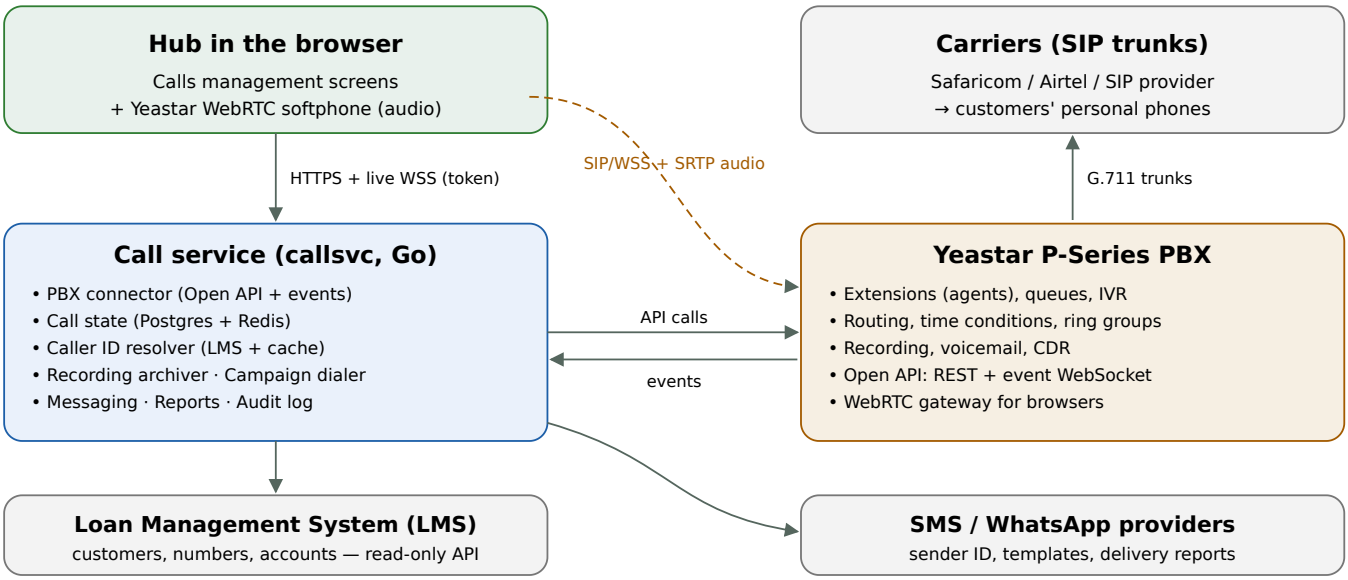



Figure 1. The whole system. Audio goes browser ↔ PBX ↔ carrier. Everything else (who is calling, state, records) goes through the call service.

| Component | Does | Never does |
|---|---|---|
| Hub frontend | Screens; the softphone (Yeastar WebRTC SDK); redraws from live events | Talks to the LMS or PBX API directly; holds PBX credentials |
| Hub backend | Sign-in, roles and rights, widget access; issues a 5-minute call-service token | Call logic |
| Call service | All call logic, state, lookups, recordings, campaigns, messaging, reports, audit | Keeps LMS data beyond a short cache |
| Yeastar PBX | Registrations, routing, IVR, queues, audio mixing, recording, CDR, trunks | Knows our customers |
| LMS | Source of truth for customers and accounts | Gets written to except agreed interaction write-backs |

## 2. VoIP from scratch

A voice call over a data network has two separate halves. **Signalling (SIP)** sets the call up, changes it and ends it. **Media (RTP, encrypted as SRTP)** carries the sound in packets of 20 ms. They use different ports and often different paths: "connected but silent" almost always means signalling worked and media was blocked.

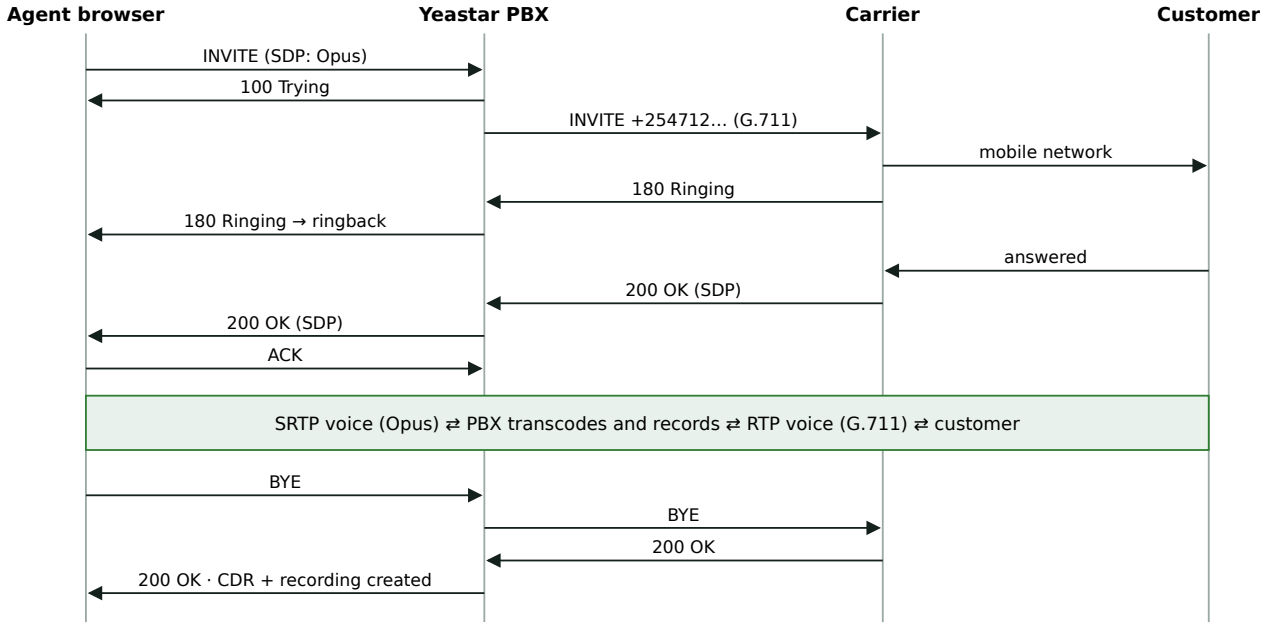



Figure 2. An outbound call, message by message. The PBX is in the middle of both legs (a back-to-back user agent), which is what lets it record, hold, transfer and monitor.

| SIP message | Meaning |
|---|---|
| `REGISTER` | "Extension 1001 is reachable here" (repeated) |
| `INVITE` / re-INVITE | Start a call / change it (hold = `sendonly`) |
| `180` / `183` | Ringing / early media (announcements, tunes) |
| `200 OK` + `ACK` | Answered; audio starts |
| `REFER` | Transfer |
| `BYE` / `CANCEL` | Hang up / before answer |
| `486` `480` `404` `603` | Busy / unavailable / no number / declined |

| Codec | Per call | Used between |
|---|---|---|
| G.711 a-law | ~87 kbps | PBX ↔ Kenyan carriers (standard) |
| Opus | 24–40 kbps | Browser ↔ PBX (WebRTC, adapts to bad networks) |
| G.729 | ~31 kbps | Only on very thin links |

**Plan 100 kbps each way per concurrent call.** The PBX transcodes Opus↔G.711 (CPU sizing). Keys pressed travel as RFC 4733 DTMF events, not beeps.

### WebRTC: Hub is the phone

1. Hub asks the call service for a **login signature**; the service creates it on the PBX (`/openapi/v1.0/sign/create`). The extension password never reaches the browser.
2. The Yeastar WebRTC SDK registers the extension with SIP over secure WebSocket (e.g. `wss://pbx…:8088`).
3. Audio flows as SRTP with DTLS keys — always encrypted in WebRTC.

4. ICE tries every network path; STUN finds the public address; a **TURN** relay (our coturn server) carries audio when firewalls block direct paths (home and corporate networks).

| Quality | Good | Poor |
|---|---|---|
| Latency (one way) | < 150 ms | > 300 ms |
| Jitter | < 30 ms | > 50 ms |
| Packet loss | < 1% | > 3% |
| MOS (1–5) | ≥ 4.0 | < 3.5 |

| Ports | Purpose |
|---|---|
| TCP 8088 (config.) | Browser ↔ PBX WebRTC signalling |
| UDP/TCP 5060, TLS 5061 | PBX ↔ carrier SIP (carrier IPs only) |
| UDP 10000–12000 | RTP/SRTP media (PBX range) |
| 3478 / 5349 + relay range | TURN |

**Trunks:** a SIP trunk from a licensed carrier gives our public numbers (DIDs), a number of **channels** (simultaneous calls; running out = busy for customers) and the right to present our numbers as caller ID. Use **two carriers** for failover. All numbers are stored in E.164 (`+254712345678`).

# 3. Architecture and trust

### Why a separate microservice

- Bursty event load and its own failure mode: a slow call component never slows Hub, and vice versa.
- PBX and LMS credentials live only here; stricter retention and audit for recordings and financial data.
- All Yeastar code sits in one adapter (PBX connector) — replaceable by Asterisk/FreePBX later.

### How access is enforced on the server

1. Agent signs in to Hub. The Calls management widget is switched on for chosen groups.
2. Hub backend issues a signed **5-minute token** (person, org, calls rights).
3. Every request and the live WebSocket carry it; the call service verifies it and checks the right for each action. Refusals are logged.

| Right | Allows |
|---|---|
| `calls.use` | Softphone, own calls, customers on own calls |
| `calls.supervise` | Team calls, wallboard, listen / whisper / barge |
| `calls.recordings.play` / `.download` | Recordings (download requires a reason) |
| `calls.campaigns.manage` / `.run` | Campaigns |
| `calls.dnc.manage`, `calls.routing.manage`, `calls.reports`, `calls.audit` | Do-not-call list, routing, reports, compliance log |

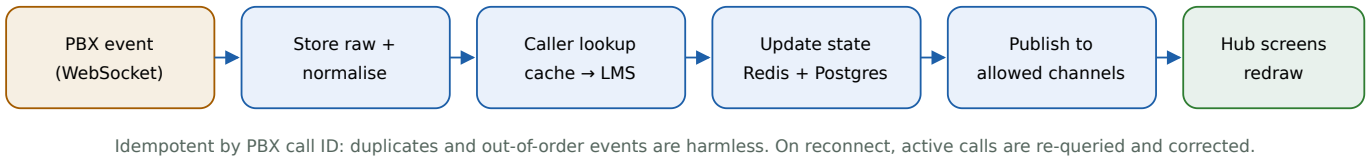



Figure 3. How a PBX event becomes what an agent and a supervisor see. Hub never treats its own view as truth; it redraws from these events.

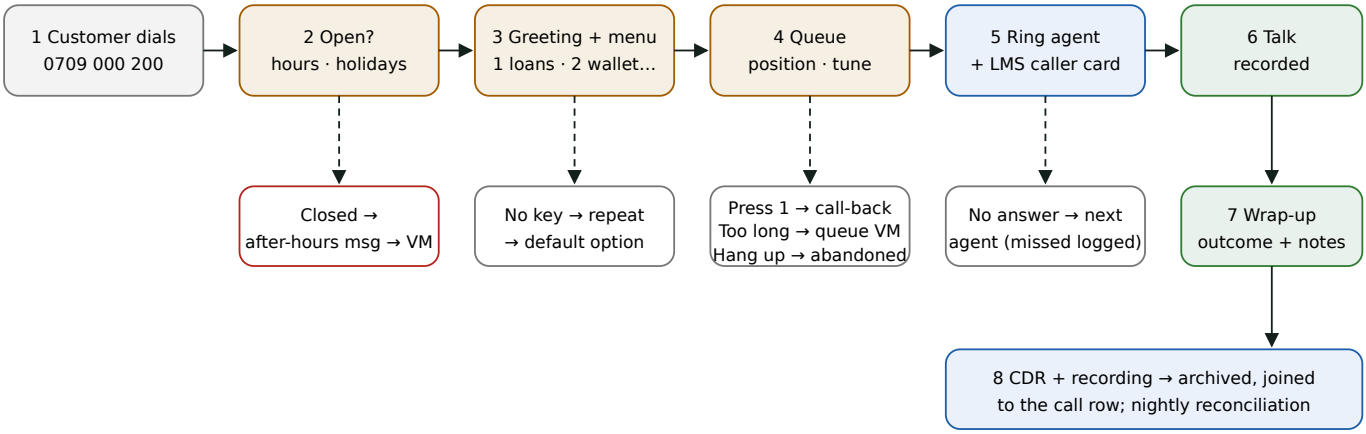
# 4. Yeastar: what we use, and what is open source

**Correction to an assumption:** only Yeastar's native mobile SDKs (`linkus-android-sdk`, `linkus-iOS-MacOS-sdk`) are published under the MIT licence. The browser SDK we need (`ys-webrtc-sdk-core`) is public on GitHub but states no open-source licence, and the PBX and its Open API are commercial. Written confirmation of our right to use them (and the plan that enables them) is a phase-0 task.

| | Browser SDK (in Hub) | Open API (in call service) |
|---|---|---|
| Controls | The agent's own calls; carries the audio | Any call; no audio |
| Confirmed functions | `call`, `answer`, `reject`, `hangup`, `hold/unhold`, `mute/unmute`, `dtmf`, `blindTransfer`, `attendedTransfer`, `startRecord`, `setSessionStaticStatus`; max **2 calls** per session; events `incoming`, `registered`, `cdrChange`; errors −106…−110 (e.g. −107 logged in elsewhere) | `verify` token (get/refresh/revoke), event WebSocket + subscribe + heartbeat, call/CDR/presence/queue events, dial (ring agent then customer), hang up, transfer, park, listen/whisper/barge, queue agent login/pause, CDR search, recording & voicemail download, config read |
| Used for | Answer, hang up, hold, mute, keys, transfers | Click-to-call via agent, campaigns, supervision, park, queues, records, archiving |

**PBX connector behaviour:** token refreshed at 80% of lifetime by one leader replica; 3 s timeout for control, 30 s for downloads; reads retried twice, **"dial" never retried** (query active calls instead, to avoid calling a customer twice); event socket heartbeat, reconnect with backoff 1→10 s, then resubscribe and reconcile active calls; every raw event stored before processing.

## 5. Inbound calls, step by step




Orange = PBX does it. Blue = call service. Green = agent in Hub. Dashed = alternative paths.

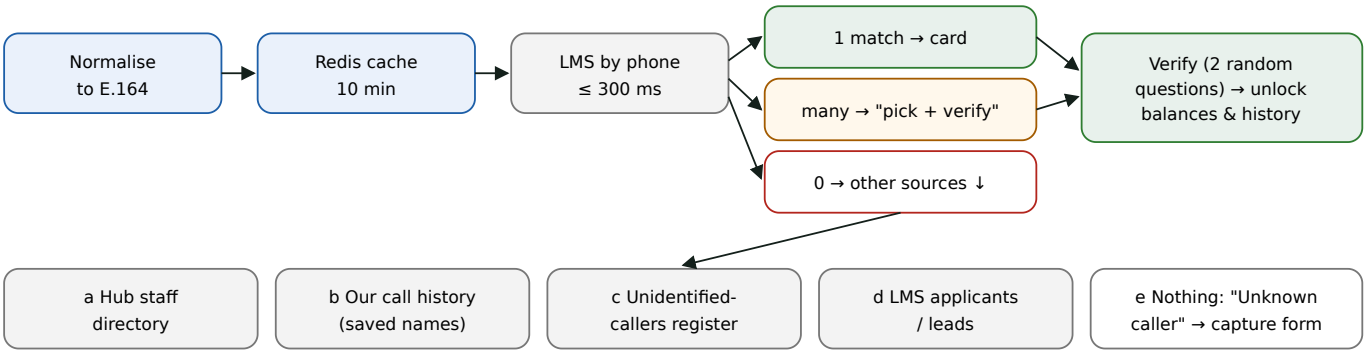
Figure 4. An inbound call's journey and every alternative path.

| The critical second (step 5) | Time |
|---|---|
| PBX rings extension 1003 and emits the call event | 0 ms |
| Connector receives and normalises (`call.ringing`) | ~50 ms |
| Redis cache checked for the caller number | ~60 ms |
| LMS lookup on a miss (hard timeout; on timeout show the number, fill the name later) | ≤ 300 ms |
| Ringing card in Hub (name, account, status, queue, wait); SDK `incoming` fires; ring tone | ~350 ms |

| Edge case | Exactly what happens |
|---|---|
| Number withheld | "Withheld number"; no lookup; identify by verification questions |
| Number shared by several customers | "Used by 2 customers" — no details until the agent picks one and verifies |
| Call service down | **Calls still ring and connect**; no names; reconciled from CDR later |
| LMS down / slow | Number shown, "lookup unavailable", retry button |
| Same customer calls 3× in 10 min | "Called 3 times today" badge |
| Fraud-flagged number | Red tag; account details hidden until verification |

# 6. Caller identification with the LMS

**A phone number identifies a phone, not a person.** Numbers are recycled, shared, ported and spoofed. So the system **identifies** (suggests who is calling, instantly) and the agent **verifies** (asks questions) before any account detail is shown.




Every name shows its source ("from LMS", "saved by an agent on 3 Nov", "staff"). Agent-typed names are grey and "unverified".
A known customer on a new number → a number-change request in the LMS process (SIM-swap protection), never edited on the call.

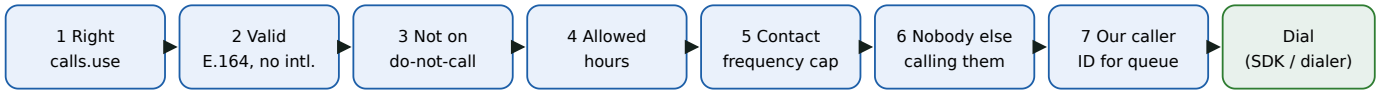
Figure 5. From a ringing number to a confident identity, including callers who are not in the LMS.

| LMS API we need (read-only + one write) | Returns |
|---|---|
| `GET /customers?phone=+2547…` | customer_id, full_name, phone_type (primary/alternate/next of kin/employer), status, do_not_contact, fraud_flag, kyc_status |
| `GET /customers/{id}/summary` | accounts, product, balance, arrears days, next due, last payment, assigned collector |
| `GET /customers/{id}/verification` | data to check answers against (never shown in full) |
| `GET /customers/search?q=` | search by name / ID / account |
| `POST /customers/{id}/interactions` | write back outcome, promise to pay (amount, date), notes |
| Mutual TLS or IP-restricted key · p95 < 200 ms (index on normalised phone) · numbers cleaned to E.164 in the LMS | |

| Field | Agent before verifying | Agent verified | Supervisor | Fraud team |
|---|---|---|---|---|
| Name, account no., product, status | ✔ | ✔ | ✔ | ✔ |
| Balance, next due, last payment, last 10 transactions | — | ✔ | ✔ | ✔ |
| Full ID number, date of birth | — | compare only | — | ✔ |
| Fraud notes | tag only | tag only | ✔ | ✔ |

Inbound calls need two passed questions even from the registered number (spoofing is cheap); outbound calls we dialled need one. Three failed verifications on an account in 24 h raise a fraud alert.

# 7. Outbound calls and campaigns




Any failed check refuses the call with a clear message and is written to the audit log.

Figure 6. Seven checks before any customer is dialled.

| Start | How |
|---|---|
| Manual / click-to-call | `POST /v1/calls` (checks) → SDK `call()` from the agent's extension; outbound route sets the chosen caller ID |
| Call-back task | From queue "press 1", missed or abandoned calls; first free agent in the queue accepts; 3 tries 15 min apart, then an SMS |
| Campaign — preview (default) | Agent sees the next customer, then presses Call |
| Campaign — progressive | When an agent is free: lock the next row → Open API dial rings the agent, then the customer → result → schedule retry after the gap |
| Predictive | **Not used** — causes silent calls to customers |

Results recorded: answered, no answer, busy, rejected, unreachable, failed, cancelled. A campaign ends for a customer on Promise to pay, Payment made, Wrong number or Do not call, or when tries run out.

## 8. Every call feature in detail

| Feature | What happens underneath | Rules |
|---|---|---|
| Mute | Browser stops sending mic audio | Recording continues |
| Hold | re-INVITE `sendonly`; PBX plays music-on-hold (care tune) | Reminder at 60 s; wallboard at 3 min |
| Keypad | RFC 4733 DTMF via PBX | Digits never stored on customer calls |
| Blind transfer | SDK `blindTransfer`; agent's leg ends | No answer → target VM or back to queue |
| Attended transfer | Consult leg with customer on hold; complete or go back | Card, verification and notes travel with the call |
| Add someone | PBX conference bridge mixes audio | Max 5 parties; join tone |
| Park | Call on slot 7xx; anyone picks up | Rings back the parker after 2 min |
| Recording | PBX "always record"; archiver downloads → SHA-256 → AES-256 → object storage → delete from PBX | **Mandatory, cannot be paused.** Plays/downloads audited; download needs a reason |
| Voicemail | Per agent and per queue; stored like recordings; optional transcript | Queue VM is a task; alert after 2 working hours |
| Agent status | Available, On a call, Wrap-up, Away/Lunch/Busy (queue pause), DND, Offline, per-queue login | Every change time-stamped for reports |
| Supervision | Listen (hear both) · Whisper (agent hears you) · Barge (all hear you) | `calls.supervise`, own teams only, all audited |
| SMS | Aggregator, approved sender ID, delivery reports, replies to thread | "STOP" → opt-out list |
| WhatsApp | Business Platform; templates to start, free text 24 h after the customer writes | Templates approved in advance |
| Wallboard | Live per queue and agent from Redis, every second | Alerts: wait, service level, wrap-up/away too long, trunks > 80%, poor line |
| Reports | From our calls table: by day/hour, queue SLA, agent handle time, outcomes, campaigns, first contact resolution | CSV export; scheduled emails later |

## 9. The call lifecycle

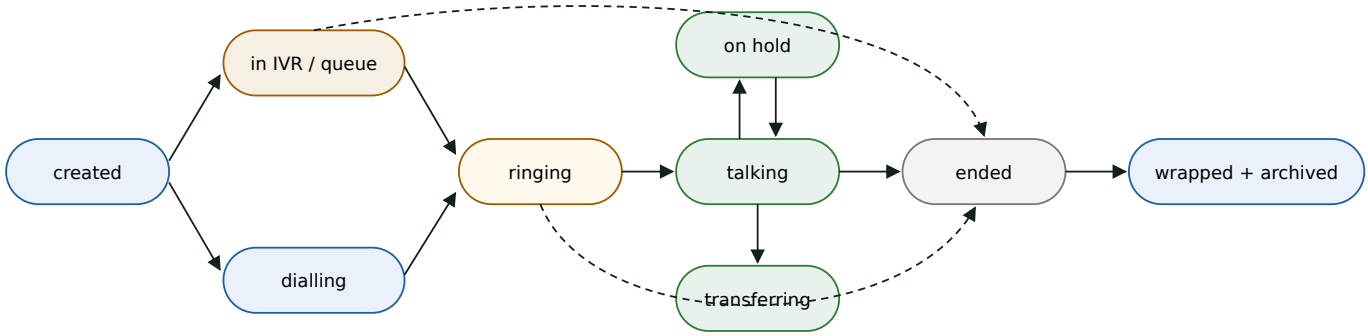



Figure 7. The states a call moves through. Timers (wait, ring, talk, hold, wrap) are derived from the transitions.

## 10. Data, API and live events

| Table | Holds |
|---|---|
| `calls` | One row per call: direction, numbers, DID, trunk, queue, IVR path, agent, customer snapshot + match source, verification, timers, result, outcome, campaign/call-back links, quality |
| `call_events`, `call_legs`, `pbx_events_raw` | Timeline, participants, raw PBX events (30 days) |
| `recordings`, `voicemails` | Storage key, checksum, encryption key id, retention, legal hold, transcript |
| `agents`, `agent_status_log` | Person ↔ extension, queues; status history |
| `callback_requests`, `campaigns`, `campaign_targets`, `dnc_numbers` | Outbound work and controls |
| `unidentified_callers`, `messages`, `routing_settings` | Non-LMS callers, SMS/WhatsApp, routing mirror |
| `audit_events` | Append-only, hash-chained compliance log |

Monthly partitions for high-volume tables (5,000 calls/day ≈ 1.8 M calls/year, ~15 events each). Key endpoints: `POST /v1/calls`, `GET /v1/calls` (paginated), `PUT /v1/calls/{id}/outcome`, `POST /v1/calls/{id}/verify`, `GET /v1/lookup`, `GET /v1/recordings/{id}/stream`, `/campaigns`, `/dnc`, `/monitor`, `/reports`, `/routing`, `/audit`. Live WebSocket `/v1/live` pushes `call.ringing`, `call.updated`, `call.ended`, `agent.status`, `queue.status`, `alert`, `callback.new`, `message.new`, with sequence numbers for catch-up after reconnect. The Hub frontend's simulated functions (marked `PBX:`) map one-to-one onto these.

## 11. Security and compliance (Kenya)

Engineering checklist, to be confirmed by our legal and compliance team.

| Source | What it requires of us here |
|---|---|
| Data Protection Act 2019 + 2021 regulations (ODPC) | Lawful basis and notice for recording; minimisation; retention limits; security; data-subject access; breach notice within 72 h; registration; a **DPIA** before go-live |
| Central Bank of Kenya (digital credit providers) | Fair debt collection and communication; no harassment |
| Communications Authority of Kenya | Numbering and caller ID rules; sender IDs; opt-out for marketing SMS |
| Consumer Protection Act 2012 | Fair treatment, accurate information |

| Encryption | |
|---|---|
| Agent ↔ PBX | SRTP + SIP over WSS (TLS 1.2+) |
| PBX ↔ carrier | SIP-TLS + SRTP if supported, else private link/VPN |
| Service ↔ LMS | Mutual TLS |
| Recordings | AES-256-GCM per file, KMS keys, yearly rotation |

| Retention (default) | |
|---|---|
| Call records, notes, messages | 7 years |
| Recordings, voicemail | 1 year unless legal hold |
| Raw PBX events | 30 days |
| LMS cache / Audit log | 10 min / 7 years |

- **Audit (tamper-evident):** calls, verifications, number reveals, recording plays/downloads, supervision, DNC/routing/campaign changes, exports, refusals. Each row hashes the previous; daily chain check.
- **Fraud:** no phone-number changes on calls (SIM-swap); random verification questions; international dialling off; PBX SIP open only to carrier IPs; night-time volume alerts; agents see only their own customers; monthly lookup review.
- **Minimisation:** numbers masked by default; snapshot not full LMS record on calls; data-subject export per customer.

## 12. Operations

| Sizing (60 agents, 5,000 calls/day) | |
|---|---|
| Peak concurrent calls | ~45 |
| Trunk channels | 60 (2 carriers × 30) |
| Bandwidth at PBX | ~10 Mbps each way |
| Recordings (compressed) | ~35 GB / month |
| Database | ~3 GB / month |

| Failure | Customers | Recovery |
|---|---|---|
| Call service down | Unaffected | Restart; reconcile CDR |
| LMS down | Unaffected | No names; retry |
| PBX down | Calls fail | PBX HA pair; carrier failover |
| One carrier down | Partial | Second trunk |
| Office internet down | Inbound fails | Carrier forwards DIDs to on-call mobiles |

Monitor: event socket up, event lag < 2 s, LMS p95 < 500 ms, registered agents, trunk use < 80%, carrier failures < 5%, recording backlog, audit chain, PBX disk. Backups: Postgres WAL + nightly (35 days, monthly restore test), recordings versioned + second region, nightly PBX config export. Targets: RTO 1 h, RPO 5 min — calls keep working meanwhile. Environments: dev (test PBX, fake LMS), staging (test trunk, masked LMS, staff phones only), production.

# 13. Rollout, decisions, and what was forgotten

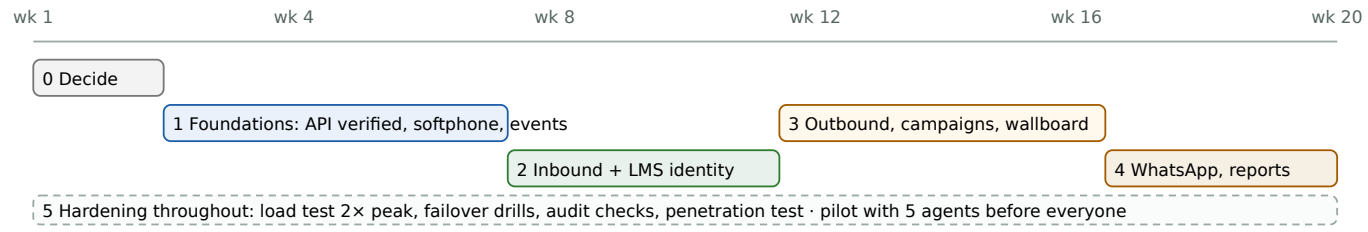



Figure 8. Indicative rollout.

## Decisions needed

1. Yeastar on-premises or Cloud edition
2. Carriers, channels, second carrier
3. Numbers per team; allowed caller IDs
4. Written Yeastar licence for Open API + WebRTC SDK
5. Headset model; home working (TURN, VPN)
6. Who builds the LMS API, and when; phone data clean-up
7. LMS write-backs (promise to pay)
8. Recording retention; refusal policy
9. Collection contact limits, hours, no-call days
10. Verification questions
11. Who may listen to recordings and supervise
12. DPIA and DPO sign-off
13. Hours, holidays, after-hours and 24/7 fraud line
14. Queues, service level targets, outcome lists

## What may have been forgotten

- **Swahili** greetings and IVR (language choice first?)
- Professionally recorded greetings, menus and hold messages
- **M-Pesa STK push** during a call (Daraja API): a promise to pay becomes a payment immediately
- Customer satisfaction survey after calls
- Speech analytics on transcripts (complaint, fraud, lawyer)
- Quality scoring of sampled calls
- Workforce planning from call history
- Flag for customers in hardship/bereavement to pause collections
- Complaints register linked to calls
- Training mode for new agents
- Number reputation with carriers (avoid "likely scam" labels)
- Mobile app for agents (Linkus SDKs, MIT)

End of handbook. Detail for each section is in the numbered Markdown files beside this PDF.